

hw09: linked lists

▼ [Hide Assignment Information](#)

Instructions

hw09: Linked lists

Focus:

- Linked Lists
- Operator Overloading
- Copy control
- Separate compilation

Attachments:

- testPolynomial.cpp
- output.txt

Problem:

You will implement a class `Polynomial` for storing and manipulating polynomial expressions. A polynomial is an expression of the form

$$a_k x^k + \dots + a_2 x^2 + a_1 x + a_0$$

The number k is called the **degree of the polynomial** and the numbers $a_0, \dots, a_k$ are called the **coefficients**.

Store each polynomial as a degree and a **singly linked list** of its coefficients with the **highest degree coefficient first**. For example, the polynomial $4x^3 + x + 7$ would be stored as:

```
head_ptr --> 4 --> 0 --> 1 --> 7
degree: 3
```

Here, the highest degree term with a coefficient of 4 comes first, then the quadratic (x^2) term, with a coefficient of 0, then the linear (x) term, with a coefficient of 1, and finally the constant term, 7. In addition, the degree of the polynomial, 3, is stored in the member variable `degree`.

The class *invariant*, i.e., the thing that must be true about the class whenever a member function is *finished* making changes, is:

1. The coefficients are stored in a linked list with the highest order term at the front of the

- list and the constant term at the end of the list. (As described above.)
2. The zero polynomial is represented by a list with a single item, zero. Note that the degree of the zero polynomial is 0.
3. The list for a non-zero polynomial does not begin with a zero coefficient. In other words, the highest degree coefficient is only zero if this is the zero polynomial.
4. The value of the degree member variable is one less than the length of the list.

What you have to handle:

- Constructors:
 - A default constructor. Obviously, the result would be a Polynomial with just a the constant term, which would be zero.
 - A constructor that takes a vector of its coefficients in order from the highest degree term to lowest
- +=
- +
- ==
- !=
 - Yes, implement it.
- <<
 - Use the caret ^ for exponentiation. So: $5x^3$ represents five times the term with an exponent of three.
 - Do not display terms with zero coefficients. Unless, of course, it is the zero polynomial. (If I didn't say that, someone would ask.)
 - Do not display coefficients whose value is 1. So you would not display $4x^3 + 1x^2$, but rather just $4x^3 + x^2$. (Unless it is the low order coefficient, i.e. the constant term.)
- A method eval that takes a single argument and evaluates the polynomial for that value of "x". So if the variable poly represents the polynomial $4x^3 + x + 7$, then poly.eval(2) would return 41.
 - You can do this in linear time, rather than quadratic time using, e.g., Horner's method.
- copy **control**, i.e. destructor, copy constructor and assignment operator

Notes

- Let's keep things simple and restrict coefficients to integers.
- **No vectors!** Aside from the one constructor that takes in a vector of the coefficients, there *should not be any use of vectors* in the entire program!
- **Do not use a linked list class.**
- No code outside of the Polynomial class needs to know about the underlying Node type, so **nest it privately**.

Test program: attached is a test program and corresponding output. Do feel free to expand on it. You may not be familiar with all of the syntax used in this program.

- **boolalpha**. This *manipulator* tells the output stream that when it is printing a bool value, true or false, to print it that way, i.e. as the strings "true" and "false". Without this manipulator, C++ will print true as 1 and false as 0.
- {2, 4, 8}. The use of braces around a comma-delimited sequence of values can be used to initialize a vector to hold that sequence of values. Please note that this use of braces requires, at least, C++14.

Do not use:

- keyword **auto**. Note that you should not use this on any assignment or exam unless the question explicitly instructs you to do so.
- The function **move**. Note that you should not use this on any assignment or exam unless the question explicitly instructs you to do so.

What to hand in:

- Header and implementation files for polynomial class, i.e. polynomial.h and polynomial.cpp.

Submit

Cancel